

## CS440 Computer Networks 2025-2026 Spring Semester

### Socket Programming Project “A Multi-user Chat Application”

Weightage: **25%**

Team size: **3-4 students** (pre-formed teams)

#### **DUE DATES:**

- A zip file containing all project files, including source code, documentation, and any additional resources (such as README file, Final Report and Presentation) on **17th May 2026 by 11:59 pm**

#### **NOTES:**

- Only one student is required to submit for the whole team.
- Please write clearly on the front page of your report all team members' names, your lab group. Marks will be deducted for submission without names or lab groups.
- Send the zip file to [tram.truong-huu@ttu.edu.vn](mailto:tram.truong-huu@ttu.edu.vn) and [huy.le@ttu.edu.vn](mailto:huy.le@ttu.edu.vn) and/or submit the file via the **TTU Courses!**:  
<https://www.google.com/search?client=safari&rls=en&q=courses+ttu+edu+vn&ie=UTF-8&oe=UTF-8>
- .

#### **A. LEARNING OUTCOMES**

This project provides a good starting point for developing a multi-user chat application using socket programming in Python. Upon completion of this project, students should be able to::

- Gain basic understanding of socket programming in Python.
- Learn to create a basic client-server application.
- Understand message passing between clients and server.
- Learn to implement multithreading or multiprocessing for handling multiple clients concurrently.
- Learn to handle errors in network applications.

#### **B. PROJECT DESCRIPTION:**

You are tasked with building a multi-user chat application in Python. The application should allow multiple users to connect to a central server and communicate with each other in real-time. The chat application optionally supports features such as user authentication, message broadcasting, private messaging, and chat room management.

**1. Server Side:**

- *Socket Initialization:* Create a server socket.
- *Client Management:* Track connected clients.
- *Message Broadcasting:* Send messages from one client to all others.
- *Client Handling:* Handle incoming and outgoing messages from each client.

**2. Client Side:**

- *Socket Initialization:* Create a client socket and connect to the server.
- *Input Handling:* Take user input and send it to the server.
- *Message Receiving:* Receive messages from the server and display them to the user.

**Detailed Steps:**

**1. Server Side:**

- Initialize a server socket.
- Listen for incoming connections.
- Accept incoming connections and store client sockets.
- Implement message broadcasting to send messages from one client to all others.

**2. Client Side:**

- Initialize a client socket and connect to the server.
- Implement input handling to take user messages and send them to the server.
- Implement message receiving to display incoming messages from the server.

**3. Features to Implement (Optional):**

- *User Authentication:* Allow users to register/login with a username and password.
- *Private Messaging:* Allow users to send private messages to specific users.
- *Chat Room Management:* Allow users to create, join, and leave different chat rooms.

**4. Testing:**

- Test the application with multiple clients to ensure proper functionality.
- Handle basic edge cases like connection drops.

**5. Documentation:**

- Write a simple documentation explaining how to run the server and connect clients.
- Provide basic instructions for using the chat application.

**6. Additional Tips:**

- Use basic socket programming without additional libraries. However, you can use libraries like ``socket``, ``threading``, or ``multiprocessing`` for handling sockets and concurrent connections.
- Prioritize simplicity over advanced features.
- Consider implementing basic error handling to deal with unexpected situations.
- Security measures such as encryption can be omitted for simplicity.

## **C. GRADING CRITERIA:**

### **1. Functionality (40%)**

- *Connection*: Application connects users to the server and facilitates message exchange.
- *Messaging*: Users can send and receive messages reliably within the chat application.
- *Core Features*: Core functionalities like message broadcasting and basic error handling are implemented effectively.

### **2. Code Quality (20%)**

- *Organization*: Codebase is well-structured and follows best coding practices.
- *Readability*: Code is clear and easy to understand, with meaningful variable names and comments.
- *Error Handling*: Basic error handling mechanisms are in place to address common scenarios.

### **3. Documentation (10%)**

- *Readme Clarity*: README provides clear instructions for setting up and using the application.
- *Completeness*: README covers all necessary information for users to interact with the application effectively.

### **4. Report (10%)**

- *Overview Clarity*: Project report offers a clear overview of the application's purpose and features.
- *Insightfulness*: Report demonstrates insight into project implementation details and challenges faced.

### **5. Presentation (10%)**

- *Content Effectiveness*: Presentation effectively showcases key features and functionalities of the chat application.
- *Organization & Engagement*: Presentation is well-structured, engaging, and encourages audience participation.

- *Q&A Interaction*: Presenter provides thoughtful responses to audience inquiries, showcasing understanding of the project's technical aspects.

6. **Concurrency (10%)**: Proper concurrency mechanisms such as multi-threading or multiprocessing should be used to handle multiple client connections concurrently.

**Total: 100%**

#### **D. SUBMISSION GUIDELINES:**

1. **Submission Format**: Submit a zip file containing all project files, including source code, documentation, and any additional resources.
2. **Additional Requirements**: Include a README file with instructions on how to run the server and connect clients, as well as any special considerations or features implemented.

#### **E. REPORT GUIDELINES:** The report should covers

1. **Project Overview**: Briefly describe the purpose and features of the chat application.
2. **Implementation Details**: Explain the design choices, algorithms, and data structures used in the implementation.
3. **Challenges**: Discuss any challenges faced during development and how they were overcome.
4. **Future Improvements**: Suggest potential enhancements or additional features that could be implemented in the future.
5. **Conclusion**: Summarize the project experience and lessons learned.

#### **F. PRESENTATION:**

1. **Format**: Prepare a brief presentation (5-10 minutes) demonstrating the functionality of the chat application.
2. **Content**: Showcase key features, user interface, and any additional functionalities implemented.
3. **Q&A Session**: Allow time for questions and feedback from the audience.

#### **G. EXAMPLE PROJECT STRUCTURE:**

---

```
|-- server.py
|-- client.py
|-- README.md
```

- |-- documentation/
  - |-- user\_guide.md
  - |-- design\_documentation.md
- |-- presentation/
  - |-- slides.pptx
- |-- report/
  - |-- project\_report.pdf

---

## **H. REFERENCES:**

Students can refer to the following links for references:

<https://www.datacamp.com/tutorial/a-complete-guide-to-socket-programming-in-python>

[https://www.tutorialspoint.com/python/python\\_socket\\_programming.htm](https://www.tutorialspoint.com/python/python_socket_programming.htm)



**Phiên bản tiếng Việt:**

**CS440 Mạng Máy Tính**  
**Học kỳ Mùa Xuân 2025 - 2026**  
**Dự án Lập trình Socket**  
**“Ứng dụng Trò chuyện Đa người dùng”**

- **Trọng số:** 25%
- **Kích thước nhóm:** 3-4 sinh viên (nhóm được thành lập trước)

**HẠN NỘP:**

Một tệp zip chứa tất cả các tệp dự án, bao gồm mã nguồn, tài liệu hướng dẫn, và các tài nguyên bổ sung (chẳng hạn như tệp README, Báo cáo cuối cùng và Bài thuyết trình) vào **ngày 17 tháng 5 năm 2026 trước 11:59 tối.**

**LƯU Ý:**

- Chỉ một sinh viên trong nhóm cần nộp bài thay cho cả nhóm.
- Vui lòng ghi rõ họ tên tất cả thành viên trong nhóm và nhóm thực hành trên trang đầu của báo cáo. Điểm sẽ bị trừ nếu không có tên hoặc nhóm thực hành.
- Gửi tệp zip đến **[tram.truong-huu@ttu.edu.vn](mailto:tram.truong-huu@ttu.edu.vn)** và **[huy.le@ttu.edu.vn](mailto:huy.le@ttu.edu.vn)** và/hoặc nộp qua **TTU Courses**:  
<https://www.google.com/search?client=safari&rls=en&q=courses+ttu+edu+vn&ie=UTF-8&oe=UTF-8>

**A. MỤC TIÊU HỌC TẬP**

Dự án này cung cấp một điểm khởi đầu tốt để phát triển một ứng dụng trò chuyện đa người dùng bằng lập trình socket trong Python. Sau khi hoàn thành dự án, sinh viên sẽ có thể:

- Hiểu cơ bản về lập trình socket trong Python.
- Tạo một ứng dụng client-server đơn giản.
- Hiểu cách truyền tin nhắn giữa client và server.
- Triển khai đa luồng hoặc đa tiến trình để xử lý nhiều client đồng thời.
- Xử lý lỗi trong các ứng dụng mạng.

**B. MÔ TẢ DỰ ÁN**

Sinh viên sẽ xây dựng một ứng dụng trò chuyện đa người dùng bằng Python. Ứng dụng này cho phép nhiều người dùng kết nối đến một máy chủ trung tâm và giao tiếp với nhau trong thời gian thực. Ứng dụng có thể hỗ trợ các tính năng tùy chọn như xác thực người dùng, phát tin nhắn, gửi tin nhắn riêng tư và quản lý phòng chat.

#### **Phía Server:**

- **Khởi tạo Socket:** Tạo một socket server.
- **Quản lý Client:** Theo dõi các client đã kết nối.
- **Phát tin nhắn:** Gửi tin nhắn từ một client đến tất cả các client khác.
- **Xử lý Client:** Nhận và gửi tin nhắn từ/đến từng client.

#### **Phía Client:**

- **Khởi tạo Socket:** Tạo một socket client và kết nối đến server.
- **Xử lý đầu vào:** Nhận dữ liệu đầu vào từ người dùng và gửi đến server.
- **Nhận tin nhắn:** Nhận tin nhắn từ server và hiển thị cho người dùng.

#### **Các bước triển khai chi tiết:**

##### **Phía Server:**

1. Khởi tạo một socket server.
2. Lắng nghe các kết nối đến.
3. Chấp nhận kết nối và lưu socket client.
4. Triển khai chức năng phát tin nhắn từ một client đến tất cả các client khác.

##### **Phía Client:**

1. Khởi tạo một socket client và kết nối đến server.
2. Xử lý đầu vào để gửi tin nhắn đến server.
3. Nhận tin nhắn từ server và hiển thị cho người dùng.

#### **Các tính năng tùy chọn:**

- **Xác thực người dùng:** Cho phép người dùng đăng ký/đăng nhập bằng tên người dùng và mật khẩu.
- **Tin nhắn riêng tư:** Cho phép người dùng gửi tin nhắn riêng cho người dùng khác.
- **Quản lý phòng chat:** Cho phép người dùng tạo, tham gia và rời các phòng chat khác nhau.

#### **Kiểm thử:**

- Kiểm tra ứng dụng với nhiều client để đảm bảo hoạt động chính xác.
- Xử lý các trường hợp ngoại lệ như mất kết nối.



**Tài liệu hướng dẫn:**

- Viết tài liệu đơn giản hướng dẫn cách chạy server và kết nối client.
- Cung cấp hướng dẫn cơ bản về cách sử dụng ứng dụng chat.

**Lưu ý bổ sung:**

- Sử dụng lập trình socket cơ bản mà không cần thư viện bổ sung. Tuy nhiên, có thể sử dụng `socket`, `threading`, hoặc `multiprocessing` để xử lý kết nối socket và xử lý đồng thời.
- Ưu tiên tính đơn giản hơn các tính năng nâng cao.
- Xem xét triển khai xử lý lỗi cơ bản để xử lý các tình huống không mong muốn.
- Các biện pháp bảo mật như mã hóa có thể bỏ qua để giữ mọi thứ đơn giản.

**C. TIÊU CHÍ CHẤM ĐIỂM**

| Tiêu chí                     | Trọng số                                                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Tính năng (40%)</b>       | Ứng dụng kết nối client-server, gửi và nhận tin nhắn thành công.                              |
| <b>Chất lượng mã (20%)</b>   | Mã được tổ chức tốt, dễ đọc, sử dụng tên biến có ý nghĩa và có chú thích đầy đủ.              |
| <b>Tài liệu (10%)</b>        | README cung cấp hướng dẫn rõ ràng về cách cài đặt và sử dụng ứng dụng.                        |
| <b>Báo cáo (10%)</b>         | Cung cấp cái nhìn tổng quan rõ ràng về ứng dụng và quá trình triển khai.                      |
| <b>Thuyết trình (10%)</b>    | Thuyết trình hiệu quả về các tính năng chính và chức năng của ứng dụng chat.                  |
| <b>Xử lý đồng thời (10%)</b> | Cơ chế đồng thời như đa luồng hoặc đa tiến trình được sử dụng để xử lý nhiều client cùng lúc. |
| <b>Tổng cộng</b>             | 100%                                                                                          |

**D. HƯỚNG DẪN NỘP BÀI**

- **Định dạng nộp bài:** Nộp một tệp ZIP chứa tất cả các tệp dự án, bao gồm mã nguồn, tài liệu và tài nguyên bổ sung.

- **Yêu cầu bổ sung:** Bao gồm một tệp README hướng dẫn cách chạy server và kết nối client, cũng như các lưu ý đặc biệt hoặc tính năng đã triển khai.

## E. HƯỚNG DẪN VIẾT BÁO CÁO

1. **Tổng quan dự án:** Mô tả ngắn gọn mục đích và tính năng của ứng dụng chat.
2. **Chi tiết triển khai:** Giải thích các lựa chọn thiết kế, thuật toán và cấu trúc dữ liệu sử dụng.
3. **Thách thức:** Thảo luận các vấn đề gặp phải trong quá trình phát triển và cách giải quyết.
4. **Cải tiến trong tương lai:** Đề xuất các tính năng bổ sung có thể được triển khai.
5. **Kết luận:** Tổng kết kinh nghiệm thực hiện dự án và những bài học rút ra.

## F. THUYẾT TRÌNH

- **Định dạng:** Chuẩn bị một bài thuyết trình ngắn (5-10 phút) trình bày về ứng dụng chat.
- **Nội dung:**
  - Giới thiệu ứng dụng.
  - Demo các tính năng chính.
  - Trình bày giao diện người dùng và các chức năng bổ sung.
  - Dành thời gian cho phần hỏi đáp và phản hồi từ khán giả.

## G. CẤU TRÚC DỰ ÁN MẪU

```
|-- server.py
|-- client.py
|-- README.md
|-- documentation/
|   |-- user_guide.md
|   |-- design_documentation.md
|-- presentation/
|   |-- slides.pptx
|-- report/
|   |-- project_report.pdf
```

## H. TÀI LIỆU THAM KHẢO

- [Hướng dẫn lập trình socket trong Python – DataCamp](#)
- [Hướng dẫn lập trình socket Python – TutorialsPoint](#)